Knowing One's Self is Wisdom: An Analysis of an LLM's Knowledge of Its Own Decoder*

Matthew Khoriaty

Project write-up, *Reliable ML with Unreliable Black Boxes*

Abstract

An autoregressive language model is trained to *predict* text, so when it *generates* it has an incentive to model its own *decoder*: the temperature, top-k cutoff, or banned-token list that turns its logits into tokens. Does the model know that decoder, and can we read it off the model's own logits, training-free? We report preliminary steps, in three parts. **(1)** The model tracks its decoder: ban a token or impose a top-k cutoff while it generates, and the model, reading back its own constrained text, lowers the right probabilities. **(2)** A streaming estimator reads an effective inverse-temperature $\beta = 1/T$ off the prefix one token at a time; on known-temperature self-text it recovers the true $1/T$ and predicts near-optimally, though natural text carries little to recalibrate (Qwen-2.5-3B is temperature-robust). **(3)** We provide two *per-sequence, distribution-free* guarantees by building that estimator as a grid ("bucket") of fixed-temperature experts, which makes it exactly Vovk's Aggregating Algorithm for log loss (Vovk, 1998). They hold on arbitrary streams: a forecast-regret bound of $\ln G$ against the best bucket, and a convex level-set error bar around the grid MLE (Lemma 1); the same construction certifies the streaming read-out of (2). Turning the read-out into better *generation* self-neutralizes and is left open.

1 Does the model know its own decoder?

At each step an autoregressive language model emits a distribution over next tokens; *decoding* chooses how to sample from it. Because models are trained to predict text rather than to generate it, a model that reads back its own generated prefix has an incentive to infer the decoder that produced it and adjust its predictions accordingly. One facet has been studied from the generation side: Mirostat (Basu et al., 2020) runs a feedback loop on the entropy of the output to hold its perplexity at a target. The question this project addresses is whether that decoder, concretely an effective inverse-temperature $\beta = 1/T$, can be *read off the model's own logits online, training-free, and with guarantees*, and whether the read-out is useful for prediction or generation. Inferring decoding settings from text is itself studied: temperature, top-k, and top-p can be *stolen* from a black-box API by querying it at scale (Naseh et al., 2023); we instead read the model's *own* logits, online and with a per-sequence guarantee. This sits at the intersection of post-hoc calibration (Guo et al., 2017; Liu et al., 2023) and the algorithms-with-predictions viewpoint (Mitzenmacher and Vassilvitskii, 2020) of the course, in which a possibly-unreliable prediction augments a downstream decision. A *worst-case, per-sequence* guarantee that holds even when the stream is misspecified is a natural thing to ask for in that setting, and Section 3 provides one.

*Code and data: `https://github.com/AMindToThink/decoding-decoding`

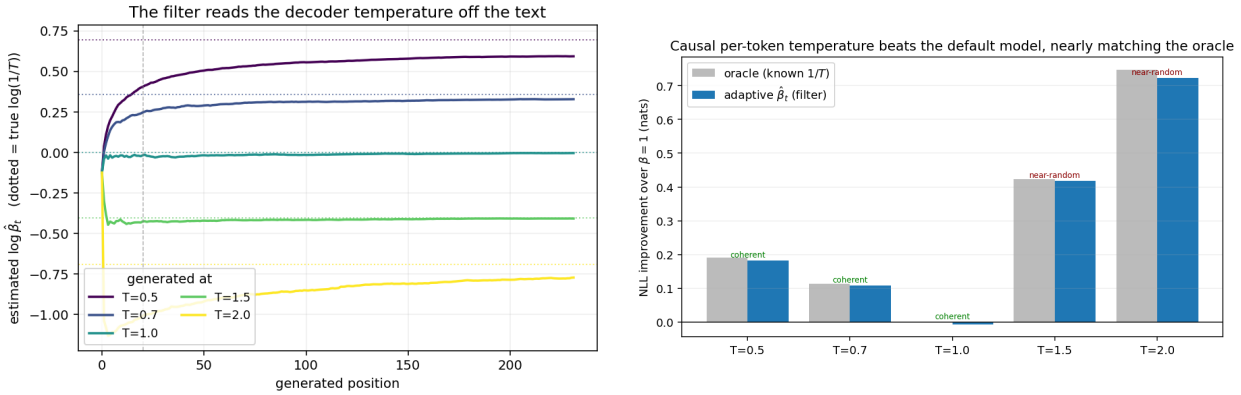

Figure 1: *Left:* on known-temperature self-text the streaming $\hat{\beta}_t$ recovers the true $\log(1/T)$ from the prefix alone. *Right:* using $\hat{\beta}_t$ as a per-token temperature lowers held-out NLL toward the oracle that knows $1/T$. (Large $T \geq 1.5$ gains are on near-random text; coherent wins are at T=0.5, 0.7.)

The model tracks its decoder. We constrain the decoder while the model generates, then ask whether the model, reading back its own constrained text, has inferred the constraint. **Blacklist:** ban a token during generation; the model lowers its predicted probability on that token across the sequence ($\Delta p_{\text{late–early}} = -0.019$ for the top token, and below an unbanned control -0.016, 95% CI $[-0.019, -0.013]$), with an early-position contrast ≈ 0 as a sanity check. **Top-k:** truncating to the top k produces no cliff at k itself; probability instead concentrates on the head, more so the tighter the cutoff. So the model tracks its decoder, though the top-k signal is broad sharpening, not a k-specific edge.

2 A streaming estimator for the effective temperature

Can we calculate what the model *should* believe its decoding temperature to be, and use that estimate to calibrate its predictions? A streaming *filter* reads an inverse-temperature $\hat{\beta}_t = 1/\hat{T}_t$ off the prefix one token at a time, training-free, using only the model's own logits.[1] The per-sequence temperature MLE this filter targets was derived independently, offline, in near-concurrent work (Mikhaylovskiy, 2026); our contribution is its *online* treatment and the per-sequence certificate of Section 3. **Recovery:** on Qwen self-text generated at a *known* temperature (64 sequences per T), $\hat{\beta}_t$ settles on the true $\log(1/T)$ from the prefix alone (bias -0.030 at T=0.7; Figure 1, left). **Prediction:** using $\hat{\beta}_t$ as a per-token temperature beats the default β=1 model by 0.108 nats at T=0.7 (up to 0.723 at T=2), within +0.025 nats of an oracle that *knows* $1/T$ (Figure 1, right). Because the filter uses only information the model itself has, it is something the model could in principle run internally to predict better. *Natural* text, by contrast, carries little to recalibrate: fitting the best global temperature across 9 types from in-distribution to far-OOD, Qwen is temperature-robust everywhere, with $T^\star \in [0.91, 1.08]$ and gain $\leq +0.0032$ nats on any real type (+0.0283 only on shuffled gibberish).

[1] A prior-regularized one-step online MLE on $\eta = \log\beta$: with a log-normal prior (mean 1, scale σ_0) and a per-token Newton/Fisher step of the categorical likelihood under $q = \text{softmax}(\ell_t)$, using $\text{score}_t = \ell_t[x_t] - \mathbb{E}_q[\ell_t]$ and $J_t = \sigma_0^{-2} + \sum_{\tau \leq t} \text{Var}_q[\ell_\tau]$ evaluated at β=1. Implementation and unit tests (shift invariance, batched = serial, recovery under matched-prior synthetic data): `src/decoding_decoding/counter_decode.py`.

What the filter lacks. This filter is fast, but its experts are *not* fixed (it re-linearizes the likelihood at β=1 each step), so it carries *no* per-sequence guarantee: on an adversarial or misspecified stream nothing bounds how wrong $\hat{\beta}_t$ can be. Supplying exactly that guarantee, and certifying this filter's outputs against it, is the subject of the next section.

3 A streaming estimator *with guarantees*: the bucket certificate

Fix a grid (a set of "buckets") of inverse-temperatures $\beta_1 < \cdots < \beta_G$, log-spaced over a bounded range. Treat each β_g as a fixed *expert* that forecasts the next token with probability $P_g(x_t) = \mathrm{softmax}(\beta_g \ell_t)[x_t]$ on the model's *raw* logits $\ell_t \in \mathbb{R}^V$ (no rescaling; fixed experts are what make the guarantees hold). Let

$$L_n(\beta) = \sum_{t \le n} \big[\mathrm{lse}(\beta \ell_t) - \beta\, \ell_t[x_t]\big], \qquad \mathrm{lse}(\cdot) = \log \textstyle\sum_v \exp(\cdot),$$

be the cumulative log loss of bucket β, let π_0 be a prior on the grid, and let the streaming forecaster predict x_t with probability $\sum_g w_g(t)\, P_g(x_t)$, where $w_g(t) \propto \pi_0[g]\, e^{-L_{t-1}(\beta_g)}$ is the Bayes posterior over buckets. This is precisely Vovk's Aggregating Algorithm for log loss (Vovk, 1998; Cesa-Bianchi and Lugosi, 2006), and the following holds for *every* logit/token sequence, with no assumption that tokens were sampled from any P_β, or from anything at all.

Lemma 1 (per-sequence certificate)**.** *For any logit sequence $\ell_{1:n}$ and any token sequence $x_{1:n}$: (i) the forecaster's cumulative log loss is $-\ln \sum_g \pi_0[g]\, e^{-L_n(\beta_g)} \le L_n(\beta_g) + \ln(1/\pi_0[g])$ for every g; with uniform π_0 the regret against the best bucket in hindsight is at most $\ln G$. (ii) With uniform π_0 the posterior mode equals $\arg\min_g L_n(\beta_g)$, the best-fit (*effective*) inverse temperature on the grid, exactly. (iii) L_n is convex with $L_n''(\beta) = \sum_t \mathrm{Var}_{q_{\beta,t}}[\ell_t]$, $q_{\beta,t} = \mathrm{softmax}(\beta \ell_t)$, so for any $c > 0$ the sub-level set $\{\beta : L_n(\beta) \le \min L_n + c\}$ is an interval containing the sequence MLE; its endpoints, read off the stored grid loss profile, are a data-dependent error bar that needs no distributional assumptions.*

Proof. (i) The forecaster's per-step log probabilities telescope:

$$\textstyle\sum_t \ln \sum_g w_g(t) P_g(x_t) = \ln \sum_g \pi_0[g]\, e^{-L_n(\beta_g)} \ge \ln\big(\pi_0[g]\, e^{-L_n(\beta_g)}\big) \text{ for each } g.$$

(ii) Immediate from $w_g(n) \propto e^{-L_n(\beta_g)}$. (iii) Each summand of L_n is an lse of an affine function of β minus an affine function, hence convex; differentiate twice. □

"Distribution-free" here does double duty. Over the data process, the lemma assumes nothing about how $x_{1:n}$ was produced. Over the estimator's own assumptions, with uniform π_0 the mode in (ii) *is* the grid MLE and the bar in (iii) is an exact profile-likelihood interval (not a Gaussian approximation); the prior survives only as the constant $c = \ln G$. We report the mode, not the posterior mean: by (ii) it equals the best-fit bucket with zero slack, whereas the mean of the posterior over a log-spaced grid (unimodal but skewed, since e^{-L_n} is log-concave by (iii)) is mildly biased.[2]

[2]Three honest caveats. First, the certificate is relative to the in-hindsight best-fit β, not the dial a counterparty actually set; under a misspecified decoder (a top-k cutoff, a blacklist) no single "true" β describes the stream, and the best-fit value is exactly what "effective temperature" should mean there. The bar then certifies *fit to the observed stream*, and a narrow bar means β is *identifiable*, not that any temperature fits well. Second, the interval in (iii) widens exactly when peaked logits make β unidentifiable ($\mathrm{Var}_q[\ell_t] \to 0$), so forecasting regret and estimation precision

4 Generating better, and limitations

Can we use the read-out to *generate* better? Not yet. The original aim was more ambitious: match the sampler's per-token temperature to a *sequence-level* target rather than a per-token one. This ran into the *label bias* problem (Andor et al., 2016), since a locally normalized per-token temperature cannot reproduce a globally normalized sequence-level distribution, and our attempts to address or circumvent it did not pan out. Even the modest closed-loop version, feeding $\hat{\beta}_t$ back into the sampler as it generates, did not help: the corrected stream comes out at effectively temperature 1, so the correction neutralizes itself (confirmed across exact-grid filters, so it is not an approximation artifact; Appendix A). We did not pin down why; one factor we believe contributes is that the model is not an optimal predictor of its own temperature. Turning the read-out into better generation is the honest open problem.

Limitations. These are preliminary steps, on a single model and tokenizer; the certificate certifies the effective β of the *observed* stream, never a counterparty's dial. The natural next step, for which it is the right tool, is misspecification: the *effective* temperature and goodness-of-fit that top-k and blacklist decoders register, where no single temperature reproduces the stream and the best-fit bucket is a projection (synthetic pilot: `scripts/e1_synthetic_pilot.py`).

A Corrector and supporting control experiments

A corrector $\beta_t^{\text{dec}} = \beta^{\text{target}}/\hat{\beta}_t^{\,p}$ divides a user's target by the estimate. In an empirical sweep it did not deliver better generation: the closed loop neutralizes itself and the output lands at effectively temperature 1. We did not establish the mechanism. Two controls in the longer draft (`paper/decoding_paper.tex`) each rule out one candidate: a sampler-switch experiment shows the belief is not a fast per-token entropy read-out, and a closed-loop-vs-invariance comparison constrains it further. The natural-text survey behind the wild-text claim of Section 2 (all 9 types) is tabulated there as well.

References

Daniel Andor, Chris Alberti, David Weiss, Aliaksei Severyn, Alessandro Presta, Kuzman Ganchev, Slav Petrov, and Michael Collins. Globally normalized transition-based neural networks, 2016. URL `http://arxiv.org/abs/1603.06042v2`.

Sourya Basu, Govardana Sachitanandam Ramachandran, Nitish Shirish Keskar, and Lav R. Varshney. Mirostat: A neural text decoding algorithm that directly controls perplexity, 2020. URL `http://arxiv.org/abs/2007.14966v2`.

Nicolo Cesa-Bianchi and Gabor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, March 2006. ISBN 9780511546921. doi: 10.1017/cbo9780511546921. URL `http://dx.doi.org/10.1017/CBO9780511546921`.

Dylan J. Foster, Satyen Kale, Haipeng Luo, Mehryar Mohri, and Karthik Sridharan. Logistic regression: The importance of being improper, 2018. URL `http://arxiv.org/abs/1803.09349v2`.

degrade in opposite directions. Third, the guarantee attaches to the mixture, not a point estimator (proper online predictors face an exp-concavity obstruction that improper mixtures avoid (Foster et al., 2018)), so transferring it to the fast filter (Section 2) is an empirical claim.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017. URL http://arxiv.org/abs/1706.04599v2.

Tong Liu, Iza Škrjanec, and Vera Demberg. Temperature-scaling surprisal estimates improve fit to human reading times – but does it do so for the "right reasons"?, 2023. URL http://arxiv.org/abs/2311.09325v2.

Nikolay Mikhaylovskiy. Estimating text temperature with language models, 2026. URL http://arxiv.org/abs/2601.02320v2.

Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with predictions, 2020. URL http://arxiv.org/abs/2006.09123v1.

Ali Naseh, Kalpesh Krishna, Mohit Iyyer, and Amir Houmansadr. Stealing the decoding algorithms of language models, 2023. URL http://arxiv.org/abs/2303.04729v4.

V Vovk. A game of prediction with expert advice. *Journal of Computer and System Sciences*, 56 (2):153–173, April 1998. ISSN 0022-0000. doi: 10.1006/jcss.1997.1556. URL http://dx.doi.org/10.1006/jcss.1997.1556.